



一些 Harness Engineering 的实践

2026-03-20  5494 发布于广东

版权

简介： Harness Engineering 是AI智能体时代的新型工程范式，核心是为Agent构建可靠环境而非优化模型。OpenAI、Anthropic、LangChain等实践表明：通过结构化知识库、双重智能体架构、组件化Harness设计及自动化反馈回路，可提升Agent在长周期、大规模任务中的稳定性与自主性。

在过去的一年里，AI智能体（Agent）的能力迎来了爆发式增长。然而，当我们将这些强大的模型投入到实际的软件工程中时，却发现它们往往难以在长时间、复杂的任务中保持稳定。为了解决这个问题，一个新的工程方法正在崛起——Harness Engineering。

什么是 Harness Engineering 呢？从 OpenAI 实践来说它是一种在构建智能体时从“编写代码”转向“构建环境”的工程方法，其核心在于设计一套环境让智能体（如 Codex）能够可靠、自主且大规模地执行软件开发全生命周期的约束架构和反馈回路。

01



OpenAI 从零开始的百万行代码实验

2025年8月下旬，OpenAI的一个工程团队开始了一项激进的实验：构建并交付一款软件产品的内部Beta版，其中没有一行代码是人工编写的。

他们从一个空的Git代码仓库开始，由Codex智能体负责编写应用逻辑、测试、CI配置、文档甚至内部工具。这一实验的目标，是理解当软件工程团队的主要工作不再是编写代码，而是设计环境、明确意图和构建反馈回路时，会发生哪些变化。



YueGuan

+关注

33

文章



9



5



2



AGENTS.md 引发的早期问题

为了解决这些问题，OpenAI的工程师们转变了角色，从“编写代码”转向了“设计系统、架构约束和复用效应”。他们提出了一系列创新的解决方法。

第一，将代码仓库定位为记录系统。在代码仓库中尽可能记录 Codex 所需的信息，由Codex生成一切，连指导智能体初始工作的“说明书”—— AGENTS.md 文件——也是由 Codex 生成的，然而 AGENTS.md 却马上成为其早期碰到的主要问题之一。

最初生成的 AGENTS.md 是一本成百上千页的巨型说明书，但大量的说明指令反直觉的没有带来如预期中的效果改进。这个问题带来一些新的经验。

经验	表现	根本
上下文（Context）稀缺	巨大的指令文件挤掉任务和代码	上下
指令失效	指令中过多规则导致模式匹配而非理解	注意
知识腐烂	手册变成陈旧规则的坟场	缺乏
难以核实	单一文本块无法进行结构化检查	缺乏

吸取早期的教训后，他们放弃了给智能体提供一本“1000页说明书”的做法，因为它会抢占掉任务、代码和相关文档这些可能更重要的信息所需的上下文窗口资源，导致智能体错过关键约束。

相反，他们将 AGENTS.md 重新定位为内容目录，指向结构化的 docs 目录，在这个目录存放代码仓库的知识库。

这样就把 AGENTS.md 从一本“百科全书”变成了“索引地图”，其大小变得仅 100 行左右，极大缩减占用的上下文窗口。



YueGuan

+关注

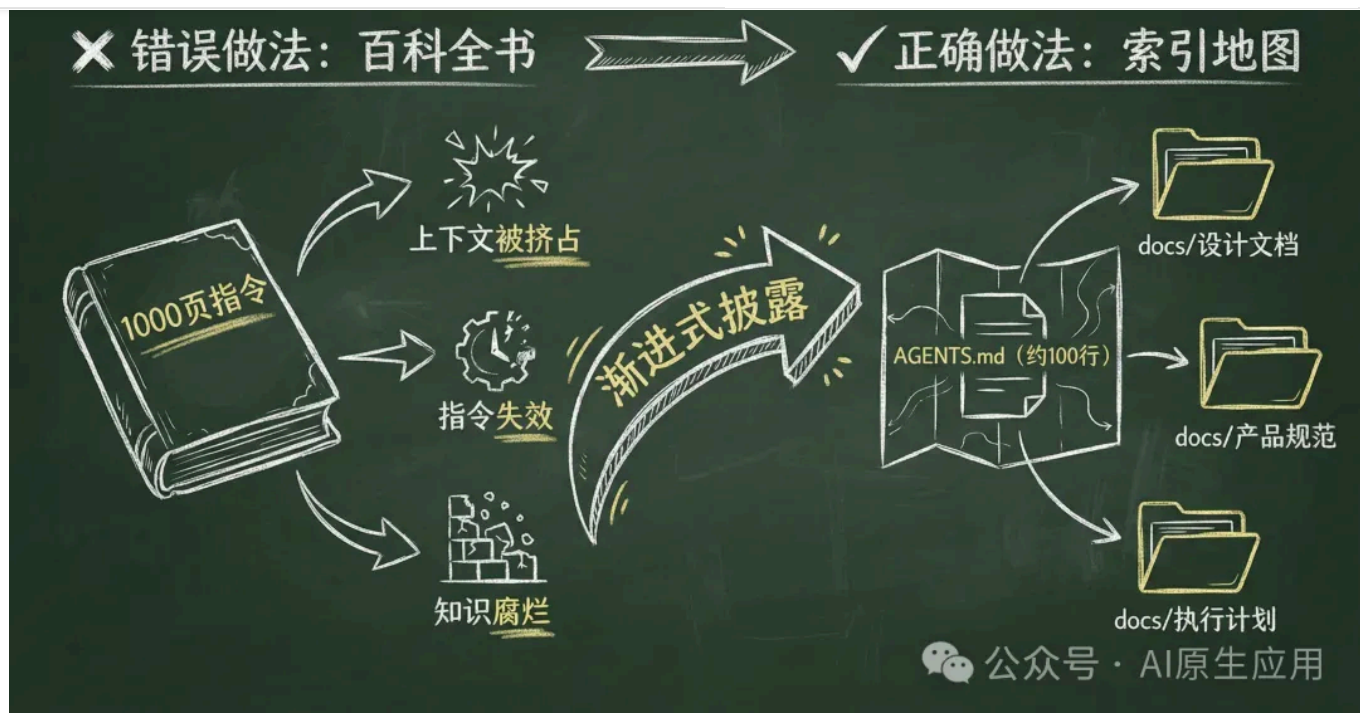
33
文章

☆ 9

👍 5

💬 2

🔗



这也实现了渐进式披露，智能体从一个小而稳定的切入点“索引地图”开始，被指导下一步该去哪里查看知识库，通过这张“地图”去知识库目录查找真实信息来源，而不是一开始就被淹没在巨量上下文窗口信息海洋里。



YueGuan

+关注

33
文章

☆ 9

👍 5

💬 2



```
2 ARCHITECTURE.md
3 docs/
4 |— design-docs/
5 |   |— index.md
6 |   |— core-beliefs.md
7 |   |— ...
8 |— exec-plans/
9 |   |— active/
10 |   |— completed/
11 |   |— tech-debt-tracker.md
12 |— generated/
13 |   |— db-schema.md
14 |— product-specs/
15 |   |— index.md
16 |   |— new-user-onboarding.md
17 |   |— ...
18 |— references/
19 |   |— design-system-reference-llms.txt
20 |   |— nixpacks-llms.txt
21 |   |— uv-llms.txt
22 |   |— ...
23 |— DESIGN.md
24 |— FRONTEND.md
25 |— PLANS.md
26 |— PRODUCT_SENSE.md
```

公众号 · AI原生应用

“索引/地图” AGENTS.md 文件内容示例

第二，提高 Agent 的可读性。 随着代码吞吐量的增加，人工QA能力成为瓶颈。他们一直在尝试让 Codex 可以直接访问 UI、日志和应用指标等数据来解决这一瓶颈。

为了让智能体能够复现问题、修复错误甚至直接推理UI行为和系统状态，他们将Chrome DevTools 协议接入智能体运行时，创建用于处理DOM快照、截图和导航的技能。

同时，提供本地可观测性堆栈（日志、指标、追踪记录），使智能体能够使用LogQL和PromQL进行查询。以上实践极大地提升了 Agent 的可读性。

存储在 Google Docs、聊

今天本地的 已版本化的



YueGuan

+关注

33
文章

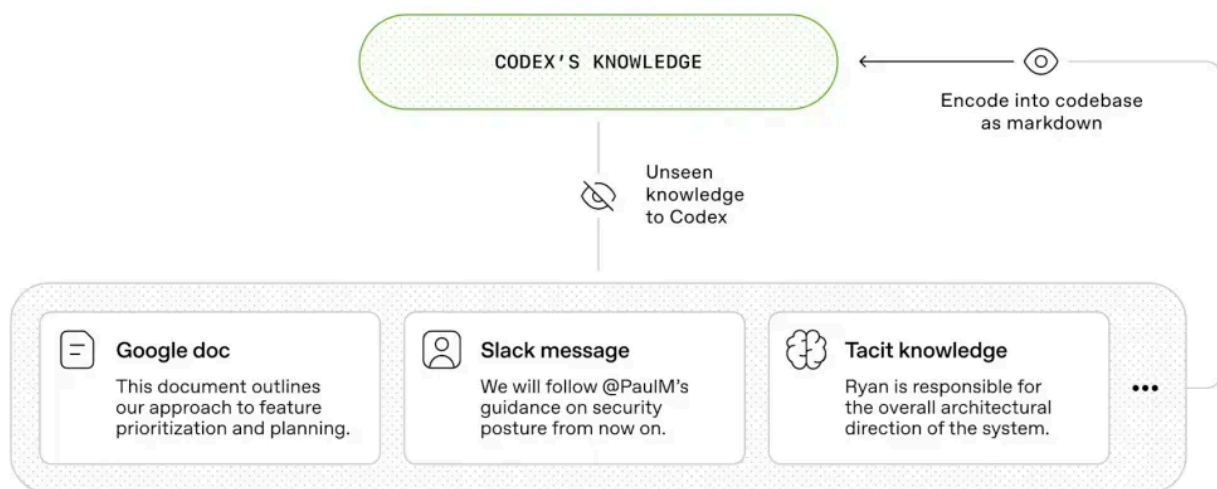
☆ 9

👍 5

💬 2



The limits of agent knowledge.
What Codex can't see doesn't exist



公众号 · AI原生应用

图源 OpenAI, 智能体知识库可见性分析框架

随着时间推移，一定会有越来越多内容存储到代码仓库中，而他们需要保证正确地组织这些信息，这样才能帮助 Agent 更好的进行推理。

他们使用了专职的 linter 和 CI 作业，验证知识库的更新状况、是否已交叉链接且结构正确。一个定期运行的“doc-gardening”智能体会扫描那些不再反映真实代码行为的过时或废弃文档，并发起修复用的 Pull Request。

第三，规范架构与品味。 仅靠文档本身，无法保持完全由智能体生成的代码库的连贯性。他们通过自定义的lint和 CI 作业，自动化强制执行严格的系统约束如本次实验中的架构模型，在架构模型中约束了编码分层固定的依赖方向。这确保了在智能体高速生成代码时，架构不会发生漂移。

具体的实践经验中，他们通过自定义的代码检查器 linter 和 CI 作业来强制执行这些系统约束规则，并增加了一些“代码品味”。例如，通过自定义 lint 静态地强制执行结构化日志记录、模式和类型的命名约定、文件大小限制，以及特定平台的可靠性要求。

由于这些 lint 是自定义的，在编写错误信息时还可以在智能体上下文中注入修复指令，形成反馈回路，这样能够帮助智能体了解本次错误的解决方法。这种思路可以把传统的碰到错误人工修复，转变为由智能体直接修复



YueGuan

+关注

33
文章

☆ 9

👍 5

💬 2



Agent 自主性带来的熵增问题

随着 Codex 自主水平的提高，新的问题出现了：代码库的熵增与垃圾收集。完全自主的智能体会复现代码仓库中已存在的模式，包括那些不均衡或不够理想的模式，随着时间的推移不可避免地导致代码库逐渐“腐烂”。

最初，团队每周花五分之一的时间通过手动方式清理“AI 腐烂残渣”，但这显然不具备可扩展性，也无法产生复用效应。

对此，他们提出了新的解决方法：将“黄金原则”（带有主观意见的自动化规则）直接编码到代码仓库中，并建立了一个循环清理流程。定期运行后台Codex任务，它会扫描偏差、更新质量等级，并发起有针对性的重构Pull Request，类似于系统的“垃圾回收”机制。

实验成果与后续

通过这一系列方法，最开始的需求场景已经取得了显著的完成度：在五个月内，该代码仓库达到了约一百万行代码，平均每位工程师每天处理3.5个PR，且产品已在数百名内测用户中投入使用。

然而，OpenAI也坦言，未来仍有问题待研究解决：在一个完全由智能体生成的系统中，不确定架构连贯性会如何随着时间的推移而演变。

他们能做的是努力学习人类工程师是通过什么方式来判断保证架构连贯性的，然后把学习到的人类能力编码提供给智能体增强它的判断力。

从工程实践来说，他们目前最棘手的挑战集中在设计环境、反馈回路和控制系统方面，以帮助智能体实现目标：大规模构建和维护复杂、可靠的软件。

02



YueGuan

+关注

33
文章

☆ 9

👍 5

💬 2



除了OpenAI，还有一些前沿 Agent 构建实践者也在Harness Engineering上积累了丰富的经验。他们的工程实践可能没有明确定义为 Harness Engineering，但从工程实践的目标来说是相似的。

Anthropic 通过双重 Agent 架构实现长期运行智能体

Anthropic在工程实践中遇到的主要挑战是：Claude在处理跨越多个上下文窗口的复杂任务时难以取得进展。

长时间运行的智能体面临的核心挑战在于，它们必须在离散的会话中工作，每个新会话开始时都对之前的工作一无所知。就像是一个大型软件项目由多个工程师轮班工作，每个新工程师都对其他人的工作内容毫无记忆。

由于 Claude 上下文窗口有限，而且大多数复杂项目无法在单个窗口内完成，智能体需要一种方法来弥合多个编码会话之间的空白。

Claude 通用智能体框架不仅擅长编码，还具备上下文管理功能如上下文压缩。理论上它应该可以在长期任务中取得不错的效果。但反直觉的是，其生成的结果难以作为生产级 web 应用。

他们发现智能体容易陷入两种失败模式：一是试图“一步到位”导致上下文耗尽；二是在项目后期过早宣布任务完成。

为此，他们设计了双重智能体架构：

- **初始化智能体 (Initializer Agent)**：在首次运行时设置环境，包括编写全面的功能需求列表（JSON格式，初始标记为“失败”）、创建 `init.sh` 脚本、一个记录任务进度的 `claude-progress.txt` 文件和初始Git提交。
- **编码智能体 (Coding Agent)**：在后续会话中，被提示每次只处理一个功能，通过阅读进度文件和Git日志快速了解现状，并在完成后提交带有描述性信息的Git 提交。

这种架构关键在于找到一种方法，让智能体在打开一个全新的上下文窗口时能够快速了解任务工作流。这些实践的灵感来源



YueGuan

+关注

33
文章

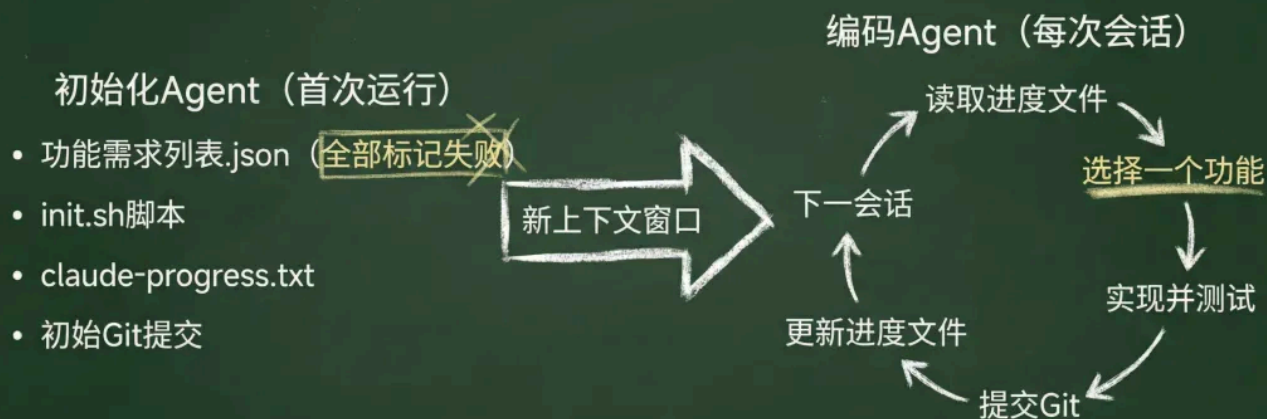
☆ 9

👍 5

💬 2



Anthropic双重Agent架构：初始化Agent + 编码Agent workflow



他们也深入讨论了环境管理。首先是要求 Agent 提供全面的JSON格式需求列表文档，这文档可以让智能体了解其所需处理的具体工作。

然后在初始化智能体搭建好初始环境框架后，下一轮的编码智能体被要求每次只处理一个功能。这种增量式方法对解决智能体倾向一次性处理过多任务的问题至关重要。

在实践中，他们发现实现增量式开发的最佳方法是要求模型使用描述性的提交信息将其开发进度提交到 Git，并在进度文件中写入进度摘要。这样，智能体就可以使用 Git 回滚错误的代码更改，并恢复代码库的正常状态，也可以明确未完成的需求任务。

此外，Anthropic强调了端到端测试的重要性，Claude 会倾向于不通过充分的测试就把某个功能标记为已完成，但是通过提供浏览器自动化工具（如Puppeteer MCP），让Claude能够像人类用户一样测试 web 应用，可以显著提高其可靠性。

LangChain 通过 Harness 让模型智能更有用

LangChain提出了一个核心公式：**Agent = Model + Harness**。



YueGuan

+关注

33
文章

9



5



2





智能体 = 模型 + 外围系统 (Harness)

模型提供智能, Harness让智能有用

公众号 · AI原生应用

Harness 是指除 Model 本身之外的所有代码、配置和执行逻辑。原始模型并非 Agent，但当 Harness 赋予它状态、工具执行、反馈循环和可强制执行的约束等功能时，它就变成了 Agent。

他们总结了 Harness 的几个关键组件：

- 系统提示
- 工具、技能、MCP+及其描述
- 捆绑式基础设施（文件系统、沙箱、浏览器）
- 编排逻辑（子Agent生成、交接、模型路由）
- 用于确定性执行的钩子/中间件（压缩、延续、代码检查）



YueGuan

+关注

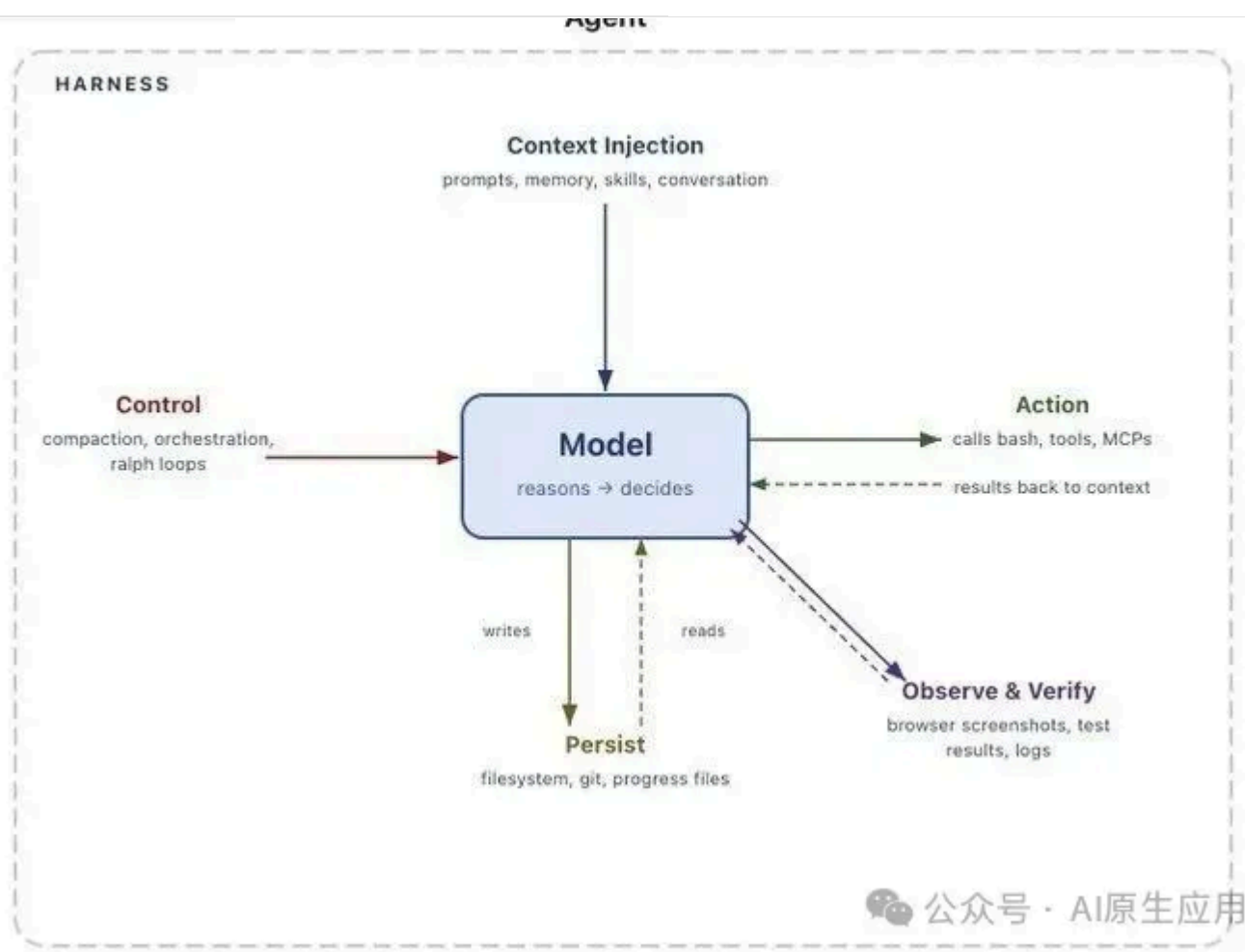
33
文章

☆ 9

👍 5

💬 2





图源 LangChain, Agent Harness 关键组件

LangChain 的工程实践并不是从一个具体的实验场景或者一个明确的挑战出发的，而是从期望 Agent 所需能力的角度来推导 Harness Engineering 的。当然这个 Agent 期望能力可能是模型现在无法提供的，于是通过 Harness 来提供能力。



YueGuan

+关注

33
文章



9



5

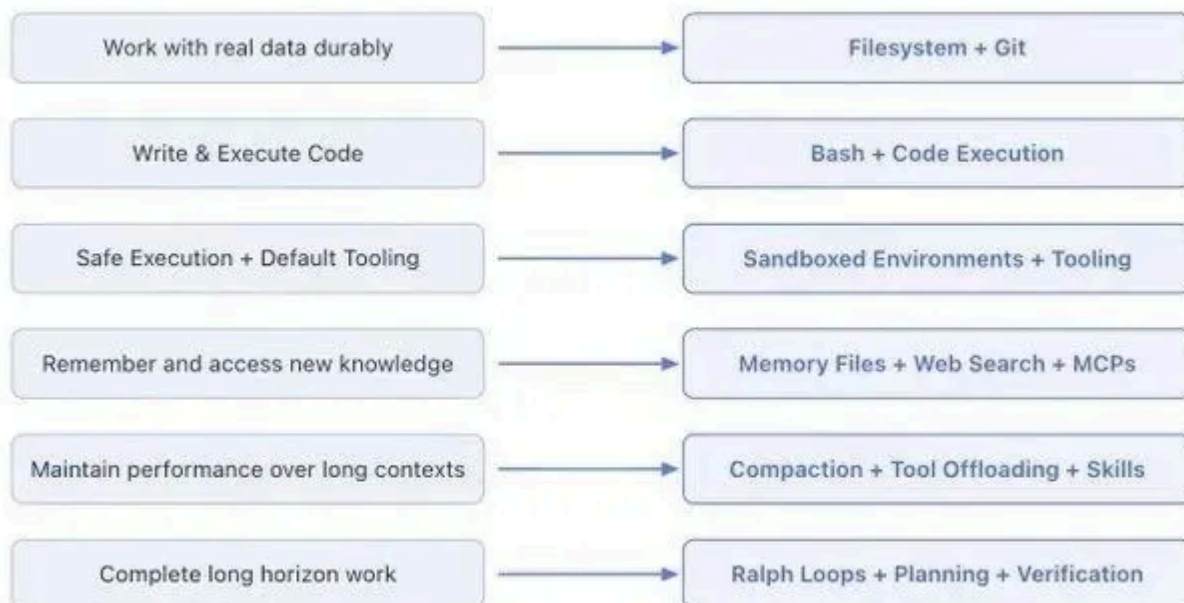


2



Desired Agent Behavior

What the Harness Adds



Each harness feature derives from a behavior the model can't deliver on its own

图源 LangChain，从期望 Agent 能力来驱动工程实践

举例来说，实现一个 Chatbot 类型产品，期望 Agent 可以追踪用户过往的聊天记录与展开新的对话。那么需要 Agent 拥有持久化存储的能力，于是在工程实践中通过文件系统来作为存储系统。

在具体的工程实践中，他们希望 Agent 拥有持久存储，以便与真实数据交互，卸载与上下文不符的信息，并在会话之间持久保存工作。通过提供文件系统抽象工具，帮助 Agent 拥有文件系统读取与管理能力。

在使用工具方面，还需提高 Agent 自主解决问题的能力，从而提升 Agent 的自主性。一种解决方法是提供 Bash 与编码能力，Agent 通过自主编码去解决遇到的问题。这样就无需人类去内置大量的工具在 Agent 中。

考虑到自主编码执行，则需要为 Agent 提供一个安全环境执行代码。在安全环境中可以提高代码执行的安全性，而沙箱作为安全环境与本地环境进行隔离是一个直接的选择。

在执行后需要观察执行结果则还需要配置对应的工具，这样提高 Agent 的可读性。如将日志与 UI



YueGuan

+关注

33
文章

☆ 9

👍 5

💬 2



务的处理进度。另外模型的预训练数据之外的知识 Agent 是不知道的，此时需要其拥有学习新知识的能力才能去实时解决问题。这一般通过上下文注入来实现，同时提供搜索工具帮助 Agent 获取实时知识去解决问题。

03

实践总结

综合这些前沿公司的实践，我们可以发现他们在Harness Engineering上面临着共同的挑战，但也提出了各有侧重的解决方案。

问题的共同点与差异

共同问题：

- 上下文限制与状态丢失：长时任务超出了单一上下文窗口的限制，导致智能体“失忆”或推理能力下降（Context Rot）。
- 任务分解与过度自信：智能体倾向于试图一次性解决复杂问题，或者在未进行充分测试的情况下过早宣布任务完成。
- 环境与工具的复杂性：随着工具数量的增加，智能体容易迷失或产生幻觉。

差异问题：

- OpenAI面临的是百万行代码级别的规模化挑战和代码库的熵增问题。



YueGuan

+关注

33
文章



9



5



2

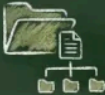


解决方法的共同点与差异

共同点：

- 依赖文件系统与版本控制：无论是OpenAI的 docs/ 目录、Anthropic的进度文件，还是LangChain对文件系统的强调，大家都认同文件系统是智能体最好的外部记忆和协作平面。
- 建立自我验证与反馈回路：通过提供浏览器、测试脚本和 Linter，让智能体能够自己运行代码、查看报错并修复问题。
- 增量式进展：强制智能体将大任务分解为小步骤，每次只专注于一个具体的Feature或PR。

三家公司实践对比

	OpenAI	Anthropic	LangChain
核心挑战	百万行代码规模化	多会话连贯性	从零推导Harness
核心方法	架构约束+垃圾回收	双重Agent+进度文件	组件化标准化
共同依赖			

文件系统是共同基础

文件系统、增量进展、自我验证

公众号 · AI原生应用

差异点：

- OpenAI更侧重于架构约束和系统级治理（如自定义Linter、垃圾回收机制）。
- Anthropic侧重于工作流的拆分（初始化与编码智能体分离）和状态的显式追踪（JSON格式的功能列表）。



YueGuan

+关注

33
文章

☆ 9

👍 5

💬 2



结语

Harness Engineering 正在让 Agent 构建从关注“模型能力”转向“工程确定性”。

OpenAI 的核心挑战已从单一的模型推理，转向环境设计、反馈回路与控制系统的协同。这传递了一个明确信号：即便模型推理能力触及天花板，若缺乏坚固的“工程脚手架”，AI 依然无法在复杂的生产场景中实现交付可靠性。

正如人类开发者离不开 IDE 和 CI/CD，Agent 同样需要属于它的基础设施。这本质上不再是单纯的模型能力博弈，而是一场关于工程架构设计的系统性重构。

(如果这篇文章对您所有帮助，请帮忙 关注 并 转发，谢谢！)

参考资料

[1] OpenAI. (2026). Harness engineering: leveraging Codex in an agent-first world.
<https://openai.com/zh-Hans-CN/index/harness-engineering/>

[2] Anthropic. (2025). Effective harnesses for long-running agents.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

[3] LangChain. (2026). The Anatomy of an Agent Harness.
<https://blog.langchain.com/the-anatomy-of-an-agent-harness/>

[4] Manus. (2025). Context Engineering for AI Agents: Lessons from Building Manus.
<https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>

文章标签：

开发工具

git

人工智能

定位技术

存储



YueGuan

+关注

33 文章

☆ 9

👍 5

💬 2

🔗

阿里云开发者 586 阅读

Qoder 工程实践：Harness Engineering 指南

Harness 是一套面向 AI Agent 的工程化框架，通过将架构约束、规范文档和自动化验证（如依...

阿里云开发者 991 阅读



深入理解OpenClaw技术架构与实现原理（上）

本文深度剖析OpenClaw——当前最热门的个人AI助手系统，涵盖其本地优先、多端联动的总体架...

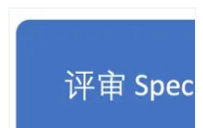
阿里云开发者 3083 阅读



打造高可靠 AI 助手：Skill 编排、Workflow 设计与 Spec Coding 的深度实践

文章首先拆解了上下文工程的五大最佳实践模式（状态管理、渐进式上下文、结构化输出、模版...

阿里云开发者 1047 阅读



2026 年 AI 编码的“渐进式 Spec”实战指南

这次分享的内容来自作者在实际项目中落地 AI 编码的一些实践和思考。希望能给正在尝试或想要尝试 AI 编码的同学...

阿里云开发者 1030 阅读

Harness 驾驭工程是 AI 平权的必经之路？

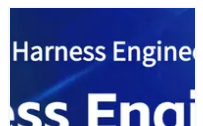
Harness Engineering 是让企业拥有一支可编排、可治理、可持续进化的数字化智能团队，CLI-Anything、HiClaw 这...

阿里云云原生 515 阅读

Harness Engineering让AI项目工程化

Harness Engineering Harness Engineering Harness Engineering

waylau 428 阅读



还在手写Skill? hermes-agent 让 Agent 自己进化能力

Hermes-agent 是 GitHub 23k+ Star 的开源项目，突破传统 Agent 依赖人工编写Aegnt Skill 的瓶颈，首创“自我进...

YueGuan 2158 阅读

热门文章 最新文章

8739



YueGuan

+关注

33
文章

☆ 9

👍 5

💬 2



4 linux下如何使用logrotate进行日志的切分

64

展开更多

相关解决方案

- 云消息队列 RabbitMQ 实践
- AI 时代的分布式多模态数据处理实践
- 流批一体 LakeHouse 架构实践
- Serverless 事件驱动架构实践
- 多源数据下的企业 AI 助手实践

更多

相关电子书

- 低代码开发师（初级）实战教程
- 冬季实战营第三期：MySQL数据库进阶实战
- 阿里巴巴DevOps 最佳实践手册

更多



YueGuan

+关注

33 文章

9

5

2

为什么选择阿里云	大模型	产品和定价	技术内容	权益	服务	关注阿里云
什么是云计算	千问大模型	全部产品	技术解决方案	免费试用	基础服务	关注阿里云公众号
全球基础设施	大模型服务	免费试用	帮助文档	解决方案免费试用	企业增值服务	或下载阿里云APP，关注云资讯，随时随地运维
	AI应用构建	产品动态	开发者社区	高校计划	迁云服务	管控云服务
		产品定价				



YueGuan

+关注

33 文章

☆ 9

👍 5

💬 2

🔗

开发者社区

首页

探索云世界

问产品

动手实践

个人

积分

消息

发布



联系我们:

4008013260

法律声明

Cookies政策

廉正举报

安全举报

联系我们

加入我们

阿里巴巴集团 淘宝网 天猫 全球速卖通 阿里巴巴国际交易市场 1688 阿里妈妈 飞猪
阿里云计算 万网 高德 UC 友盟 优酷 钉钉 支付宝 达摩院 淘宝海外 阿里云盘 淘宝闪购

© 2009-现在 Aliyun.com 版权所有 增值电信业务经营许可证: 浙B2-20080101 域名注册服务机构许可: 浙D3-20210002



浙公网安备 33010602009975号 浙B2-20080101-4



YueGuan

+关注

33
文章

☆ 9

👍 5

💬 2





YueGuan

+关注

33
文章



9



5



2

